

Hardware Security Project

Charlie Crossan, Daniel Obolensky, Ryan Ames, Lucas Ding

For this project, we used OpenAI’s GPT-4.1 model. The total cost of use was around 10 cents.
Responsibilities:

- Charlie Crossan (662048241): Task 1 taxonomy and safety statement, wrote the regex capabilities for Task 5
- Daniel Obolensky (662047553): Task 2 single-sample RTL analysis, developed output code for Task 5
- Ryan Ames (6625055354): Task 3 cross-sample comparison, analyzed output of AI model to find common structures for Task 5
- Lucas Ding (662036801): Task 4 validation and testing, analyzed the output of Task 5, and improved on it

Task 1: Vulnerability Taxonomy & Safety Statement

The GHOST system targets four vulnerability categories (T1–T4), each representing a distinct class of hardware Trojan attack at the register level.

ID	Type	Description	Example from Outputs
T1	Change Functionality	Subtly alters logic to produce incorrect results under specific trigger conditions.	AES S-box output changed to 0xFF after 10 clock cycles; UART bit flipped on 10th transmission.
T2	Leak Information	Exfiltrates internal state through a covert output channel.	Shift register contents are transmitted in the trojan_leak_out pin using a 3-bit secret key trigger.
T3	Denial of Service	Forces the circuit into a hung or non-responsive state.	AES S-box output forced to 0x00 for two cycles after magic byte sequence 8'hAA -> 8'h55 -> 8'hFF.
T4	Performance Degradation	Degrades circuit speed or resource usage without full failure.	32-bit LFSR activates after 50,000 occurrences of input 8'h7B, wasting resources.

Safety Statement

This project is defensive and educational only. We only analyze toy RTL, instructor-provided sample outputs, and locally authorized code. We do not insert Trojans into real third-party designs, do not run live Trojan generation without explicit instructor authorization, and do not commit API keys, tokens, secrets, hidden bugs, private SoC information, or non-public course artifacts.

Defensive Review Rules by Vulnerability Type

One rule to protect against T1, Change Functionality, is to flag trigger-gated changes to architecturally visible outputs or control signals. In practice, this means reviewing logic that changes normal output values only after rare constants, hidden counters, unusual state variables, or unexplained input sequences appear.

One rule to protect against T2, Leak Information, is to flag any path that moves sensitive internal state to an external, debug, status, or otherwise unexpected output. This includes newly added output pins, covert leak signals, key-triggered leak state machines, or assignments from internal registers to outputs that are not required by the clean design.

One rule to protect against T3, Denial of Service, is to flag trigger-gated logic that forces reset, stall, busy, zero output, disabled output, or unrecoverable state. Normal reset logic is not suspicious by itself; the suspicious case is when reset-like behavior depends on a rare input sequence, hidden state machine, magic constant, or unexplained counter.

One rule to protect against T4, Performance Degradation, is to flag counters, shift registers, accumulators, or delay logic that consume cycles or switching activity without contributing to the module's intended output. This is especially suspicious when the extra activity is enabled by a rare trigger but does not change functional correctness, because performance Trojans often hide behind "the output is still correct."

Task 2: Single-Sample Analysis

Selected sample: `aes_sbox_HT3_gpt-4.1_A1.v` (T3 Denial of Service). This file was chosen because it clearly separates the trigger state machine from the payload override, making it a good example for explaining trigger/payload behavior at the RTL level.

Field	Detail
Original Module	<code>aes_sbox</code> ; combinational AES SubBytes S-box lookup
Modified Signals	Added: <code>clk</code> and <code>rst_n</code> inputs; added <code>seq_counter[3:0]</code> , <code>disable_counter[1:0]</code> , and <code>trojan_active</code>
Trigger Condition	The trigger is a sequential three-byte pattern detector. When <code>data_in</code> is

	8'hAA, seq_counter advances to state 1. If the next cycle is 8'h55, it advances to state 2. If the next cycle is 8'hFF, it advances to state 3. On the following clock edge, state 3 arms trojan_active and loads disable_counter.
Payload Effect	When trojan_active is asserted, the combinational output path is overridden and data_out is forced to 8'h00. This creates a denial-of-service window because the S-box no longer returns the expected substitution value. The taxonomy description says this lasts two cycles, but the exact visible duration should be verified from simulation because trojan_active and disable_counter update on clock edges.
Stealthiness	Trigger requires a rare 3-byte ordered sequence; random or incomplete tests are unlikely to intentionally exercise this exact ordered sequence, which makes the trigger easy to miss without targeted testing.
RTL Location	Lines 16-54 (trojan_insertion_begin/end blocks); a new always block was added.

Key Trojan trigger state-machine excerpt:

```
// trojan_insertion_begin //
case (seq_counter)
  0: seq_counter <= (data_in == 8'hAA) ? 1 : 0;
  1: seq_counter <= (data_in == 8'h55) ? 2 : 0;
  2: seq_counter <= (data_in == 8'hFF) ? 3 : 0;
  3: begin
      trojan_active <= 1'b1;
      disable_counter <= 2'd2;
      seq_counter <= 0;
    end
  default: seq_counter <= 0;
endcase

// trojan_insertion_end //
// Payload (lines 50-52):
if (trojan_active) data_out = 8'h00; // DoS: output stuck at zero
```

Task 3: Cross-Sample Comparison

When starting task 3, the team ran into an issue with the analysis cell, which was initially affected by a filename-parsing bug. This bug caused all 8 samples to be classified as 'Unknown' and ruined all outputs. After applying a fix, the per-type breakdown was recovered correctly, allowing for proper analysis of the trojans:

Type	Vulnerability	Avg Lines	Avg If Stmts	Avg Regs	Avg Counters
HT1	Change of Functionality	55.5	4.5	3.5	1.0
HT2	Leak Information	79.5	6.0	6.0	1.5
HT3	Denial of Service	72.0	7.0	5.0	1.5
HT4	Performance Degradation	57.0	6.0	5.0	1.5

Key Insights

When analyzing the table, the team found that HT2 generates the most complex code with an average of 79.5 lines of code with 6.0 registers on average. The team believes this is because leakage Trojans require additional covert output logic, state machines, and transmission logic. On the other hand, HT1 is the simplest, with an average of 55.5 lines of code containing 3.5 registers on average. HT3 uses the most conditional logic with an average of 7.0 if statements. This is due to its multi-step sequential trigger. HT2, HT3, and HT4 average 1.5 counters, while HT1 averages 1.0. This suggests that the generated samples frequently use counter/state-based trigger logic across multiple vulnerability types, although the exact amount depends on the vulnerability category. The generated summary also assigns UART a higher complexity score than AES, 95.2 versus 70.0. We interpret this as a design-level complexity score, not raw line count. UART is stateful and already contains transmission control logic, which gives the inserted Trojan logic more places to hide than the small AES lookup example.

Task 4: Validation & Testing

The notebook validation cell reported results for 5 samples. These 5 samples passed module/endmodule structure checks, syntax checks, and marker-based Trojan detection checks. The remaining 3 UART samples were not included in the notebook's original validation table, so they were checked manually with a Verilog syntax compiler. Therefore, all 8 batch samples have syntax-validation evidence, but only 5 are shown in the notebook table below. This validation proves that the RTL is parseable and structurally valid; it does not by itself prove the trigger/payload behavior without behavioral simulation.

File	Module Struc.	Syntax	Trojan?	Suspicious Patt.
aes_sbox_HT1	PASS	PASS	YES	0
aes_sbox_HT2	PASS	PASS	YES	2
aes_sbox_HT3	PASS	PASS	YES	4
aes_sbox_HT4	PASS	PASS	YES	4
uart_controller_HT1	PASS	PASS	YES	4

All 5 notebook-reported designs had a valid module/endmodule structure, passed iverilog syntax compilation, and contained detectable Trojan markers. The manually checked UART files should be included in the final validation log before submission.

Task 5: Mini Defense (detector.py)

A simple pattern-matching detector was implemented in detector.py. It applies 22 regex rules derived directly from patterns observed in our outputs, organized into HIGH and MEDIUM severity tiers.

Detection Rules Summary

Category	Example Rule	Severity	Target Type
Trojan markers	trojan_insertion_begin/end comment	HIGH	Any
Named signals	Signal name contains 'trojan'	HIGH	Any
T1 triggers	trojan_counter, trojan_triggered signals	HIGH	T1
T2 channels	trojan_leak, trojan_key, leak_bit_cnt	HIGH	T2
T3 sequences	Magic bytes 8'hAA, 8'h55, 8'hFF	HIGH	T3
T3 payload	data_out forced to 8'h00	HIGH	T3
T4 triggers	Rare value 8'h7B comparison	HIGH	T4
Generic	Hardcoded multi-bit magic constants	MEDIUM	T1/T4
Generic	Conditional trigger on a specific bit pattern	MEDIUM	Any

Evaluation Results

File	Flagged?	HIGH	MEDIUM	Correct?
aes_sbox_HT1	YES	26	1	TRUE POSITIVE
aes_sbox_HT2	YES	49	1	TRUE POSITIVE
aes_sbox_HT3	YES	35	0	TRUE POSITIVE

aes_sbox_HT4	YES	35	1	TRUE POSITIVE
uart_controller_HT1	YES	29	0	TRUE POSITIVE
uart_controller_HT2	YES	69	0	TRUE POSITIVE
uart_controller_HT3	YES	49	1	TRUE POSITIVE
uart_controller_HT4	YES	39	1	TRUE POSITIVE
TOTALS on Trojaned Samples	8/8 flagged	331	5	8 TP / 0 FN on this Trojaned set

Clean Dataset	Files	Flagged	Result
Batch_trojan_designs	2	1	1 false positive
Demo_designs	2	1	1 false positive
Total clean baseline	4	2	2 FP / 2 TN

False Positive / False Negative

False Positive Risk: The detector can flag normal reset/default-zero logic as suspicious because such code can look similar to a denial-of-service payload. Clean RTL often assigns outputs or registers to zero during reset or default cases, but our simple regex detector does not understand whether that zero assignment is normal reset behavior or trigger-gated malicious behavior. The generic `trojan` keyword rule also inflates the finding count on Trojaned samples because it fires on comments, marker blocks, and variable names.

False Negative Risk: The detector relies heavily on the GHOST naming convention. This means that anyone who avoids using these patterns (using uncommon variable names, hiding counters in other bits of logic, etc.) could avoid being caught. Maybe they only trigger MEDIUM level rules on purpose to bring up only a minor amount of suspicion.